

学号：202300130183	姓名：宋浩宇
实验名称：预训练语言模型基础实验——基于 GPT 与 BERT 的文本理解	
1. 实验目的	
- 理解预训练语言模型（Pre-trained Language Models）的基本概念。 - 掌握 GPT 与 BERT 的基本区别。 - 学习使用现成模型进行文本处理任务。 - 体验大语言模型在实际任务中的效果。 - 分析模型输出结果及其局限性。	
2. 实验原理	
预训练语言模型通过在大规模语料上进行训练，学习语言的统计规律。 代表模型包括： - BERT：双向编码器，适理解任务 - GPT：自回归模型，适合生成任务 例如： BERT 任务（理解） 句子分类、相似度判断 GPT 任务（生成） 文本续写、问答 这些模型可以直接用于下游任务，无需从零训练。	
3. 实验步骤与代码	
实验步骤： - 导入 transformers 库 - 加载预训练模型（如 bert-base / gpt2） - 输入测试文本 - 调用模型进行推理 - 输出模型结果 - 修改输入观察输出变化 - 分析模型行为	
代码： <pre># -*- coding: utf-8 -*- from pathlib import Path import torch DATA_PATH = r"./exp-6.2-data.txt" # BERT 多语言情感(星级 1-5, 属于在 BERT 编码器上微调的下游分类模型) BERT_SENTIMENT_MODEL = "nlptown/bert-base-multilingual-uncased-sentiment" # 中文因果语言模型续写 GPT2_CHINESE_MODEL = "uer/gpt2-chinese-cluecorpussmall" def parse_data_file(path): text = Path(path).read_text(encoding="utf-8")</pre>	

```

cls_lines, gen_prefix = [], ""
mode = None
for raw in text.splitlines():
    line = raw.strip()
    if not line:
        continue
    if line == "[分类]":
        mode = "cls"
        continue
    if line == "[续写前缀]":
        mode = "gen"
        continue
    if mode == "cls":
        cls_lines.append(line)
    elif mode == "gen":
        gen_prefix = line
return cls_lines, gen_prefix
def load_sections_fallback(path):
    lines = [ln.strip() for ln in
Path(path).read_text(encoding="utf-8").splitlines() if ln.strip()]
    if not lines:
        return [], ""
    return lines[:-1], lines[-1]
def build_sentiment_pipeline():
    from transformers import pipeline
    device = 0 if torch.cuda.is_available() else -1
    return pipeline(
        "sentiment-analysis",
        model=BERT_SENTIMENT_MODEL,
        tokenizer=BERT_SENTIMENT_MODEL,
        device=device,
    )
def run_sentiment_batch(clf, title, sentences):
    if not sentences:
        return
    print(f"\n--- {title} ---")
    for s in sentences:
        r = clf(s[:512])[0]
        label = r.get("label", "")
        score = float(r.get("score", 0.0))
        print(f"  输入: {s}")
        print(f"  输出: label={label}  score={score:.6f}")
def run_gpt2_generate(prefix, max_new_tokens=40):
    from transformers import AutoModelForCausalLM, AutoTokenizer

```

```

print("\n--- 中文 GPT2 续写 ---")
print(f"模型: {GPT2_CHINESE_MODEL}")
tok = AutoTokenizer.from_pretrained(GPT2_CHINESE_MODEL,
trust_remote_code=True)
model =
AutoModelForCausalLM.from_pretrained(GPT2_CHINESE_MODEL,
trust_remote_code=True)
model.eval()
device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
model.to(device)
if not prefix:
    print(" (未配置续写前缀, 跳过)")
    return
enc = tok(prefix, return_tensors="pt")
enc = {k: v.to(device) for k, v in enc.items()}
with torch.no_grad():
    out = model.generate(
        **enc,
        max_new_tokens=max_new_tokens,
        do_sample=True,
        top_k=50,
        top_p=0.95,
        temperature=0.9,
        pad_token_id=tok.eos_token_id,
        eos_token_id=tok.eos_token_id,
    )
text = tok.decode(out[0], skip_special_tokens=True)
print(f" 前缀: {prefix}")
print(f" 续写全文: {text}")
def main():
    print(f"数据文件: {DATA_PATH}")
    print(f"情感模型: {BERT_SENTIMENT_MODEL}")
    raw = Path(DATA_PATH).read_text(encoding="utf-8")
    if "[分类]" in raw and "[续写前缀]" in raw:
        cls_lines, gen_prefix = parse_data_file(DATA_PATH)
    else:
        cls_lines, gen_prefix = load_sections_fallback(DATA_PATH)

```

```

extra = [
    "服务态度不错, 还会再来.",
    "排队两小时, 体验非常糟糕.",
]
try:

```

```

        clf = build_sentiment_pipeline()
    except Exception as exc:
        print("\n 情感 pipeline 初始化失败, 以下为常见原因与处理建议。")
        print(f"异常类型: {type(exc).__name__} 信息: {exc}")
        print("可尝试设置环境变量")
        HF_ENDPOINT=https://hf-mirror.com 或在可联网环境重新运行。")
        clf = None
    if clf is not None:
        run_sentiment_batch(clf, "BERT 多语言情感分类(任务书示例与数据文件)", cls_lines)
        run_sentiment_batch(clf, "改变输入后的对比(任务书步骤 6)",
extra)
    try:
        run_gpt2_generate(gen_prefix)
    except Exception as exc:
        print("\n 中文 GPT2 加载或生成失败。")
        print(f"异常类型: {type(exc).__name__} 信息: {exc}")
        print("可尝试设置镜像或更换网络后重试。")
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为：

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业\第六周实验\实验十二-预训练语言模型实验——GPT 与 BERT 应用.py

数据文件: ./exp-6.2-data.txt

情感模型: nlptown/bert-base-multilingual-uncased-sentiment

--- BERT 多语言情感分类（任务书示例与数据文件） ---

输入: 这部电影很好看。

输出: label=5 stars score=0.535430

输入: 这部电影太差了。

输出: label=2 stars score=0.452745

--- 改变输入后的对比（任务书步骤 6） ---

输入: 服务态度不错, 还会再来。

输出: label=3 stars score=0.302582

输入: 排队两小时, 体验非常糟糕。

输出: label=2 stars score=0.433989

--- 中文 GPT2 续写 ---

模型: uer/gpt2-chinese-cluecorpussmall

前缀: 今天天气很好, 我决定

解码后整段(无前缀空格伪影): 今天天气很好, 我决定是不回去了。如果你对这些事情有所期待, 那么请你和她们聊聊一个月后的未来, 因为大家 仅模型续写部分: 是不回去了。如果你对这些事情有所期待, 那么请你和她们聊聊一个月后的未来, 因为大家 进程已结束, 退出代码为 0
--

图表结果为:
无图表结果

5. 结果分析

1. GPT 和 BERT 在任务处理上有什么区别? BERT 一类双向编码器, 适合在完整句子上做理解型任务, 例如分类、相似度、序列标注, 训练目标常为掩码语言建模等, 可同时利用左右上下文。GPT 一类自回归模型, 训练与推理沿序列从左到右条件生成, 更适合续写、对话等生成任务。本实验中情感脚本使用带分类头的 BERT 族多语言模型, 续写脚本使用中文 GPT2 因果模型, 与上述分工一致。
2. 模型输出是否符合语义直觉? 在可成功下载并推理的前提下, 任务书给出的明显褒义和贬义短句, 情感模型多数时候会给出与直觉相近的极性, 但多语言星级标签可能与中文口语褒贬不是一一对应, 需看你打印的 label 与 score。续写结果受采样参数影响, 可能出现重复、跑题或事实错误, 不一定句句符合常识, 应以你一次具体生成为准。
3. 改变输入文本是否会显著影响输出? 会。分类模型对整句词面与句式敏感, 情感极性词、否定、讽刺等都会改变 logits。生成模型对前缀词面高度敏感, 前缀不同则后续分布整体改变。本仓库脚本在「改变输入后的对比」一节中增加了两句额外中文评论, 你可将它们与任务书两句对比查看标签变化。
4. 模型是否会产生错误或不合理结果? 会。预训练加微调仍受训练数据偏见、领域不匹配、细粒度反讽与否定范围等因素影响, 可能错分。生成模型可能产生事实幻觉、逻辑断裂或与前缀语义不一致的续写, 需要人工或额外约束与检索手段纠偏。
5. 预训练模型相比传统方法有哪些优势? 在大规模语料上自监督预训练后, 模型携带通用语言统计与部分语义知识, 下游用少量标注即可微调或直接用开源头做推理, 特征工程成本相对传统词袋、手工模板方法更低, 对同义改写与一定噪声更稳健。缺点是算力与存储开销大, 可解释性弱, 且仍会出现上面提到的错误类型。

6. 实验总结

本次实验从数据文件读入若干待分类中文句与一个续写前缀, 使用 transformers 分别加载 BERT 族多语言情感模型做句级情感推理并打印标签与置信度, 再加载中文 GPT2 对前缀做采样续写并打印完整生成文本, 另对替换输入句重复分类以观察输出变化, 在依赖齐全且可拉取预训练权重的环境下程序可正常跑通并输出对比结果。 LLM 使用情况: 使用 LLM 辅助完成代码编写
--